SE3002 Assignment 01: Quality Evaluation of AI-Generated Software

Project: agentMom: Broadcasting / Multicasting / Secured Communication in Multi-Agent Systems

Team members: M. Bilal Tahir (24i-3166) and Taimoor Shaukat (24i-3015) **Section:** SE-B

Part 1: Requirement Scope and AI Assumptions

We selected the agentMom SRS, "Applying Broadcasting/Multicasting/Secured Communication to agentMom in Multi-Agent Systems" (version 1.1). The ten requirements below are seven functional requirements and three non-functional requirements. Together they cover the three communication modes the project is built around (unicast, multicast, broadcast), the security layer on top of them, and the architectural choice underneath them.

Section 3.2 of the SRS contains twenty-five numbered sub-requirements, and all twenty-five are covered by the ten rows below, so nothing in the specific requirements section is left out of scope; thirteen are marked as Driving Requirements to be demonstrated by phase II, and every Driving Requirement claim in the table has been checked against the SRS individually. The assignment allows complete management of one entity to count as a single FR, and each FR row is built around one entity: the unicast message (FR1), the agent's membership in a group (FR2), the multicast message (FR3), the broadcast message (FR4), the security state of a unicast message (FR5), the security state of a multicast message (FR6), and the conversation control architecture (FR7); sub-requirements describing the lifecycle or configurable attributes of the same entity are grouped into one row on that basis, not for convenience of arriving at seven.

None of the seven FRs is a pure CRUD operation, since the underlying SRS describes a messaging framework rather than a data-management system. Even on the strictest reading (treating join/leave as create and delete of a membership record), that would make one of seven FRs a CRUD requirement, comfortably inside the limit of three; on our reading it is not CRUD at all, since the requirement's substance is the delivery decision it drives, not the storage of a record. Every assumption in the Defence / basis column carries exactly one of the three labels the assignment permits (supported by the SRS, justified by an explicit design decision, or unsupported); where a behaviour rested on more than one assumption it was split into separately numbered sub-assumptions so each carries one unambiguous label, and no unsupported assumption is hidden inside a supported one.

Part 1 requirement table: 7 functional and 3 non-functional requirements

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
3.2.1.1 – 3.2.1.4	FR	Unicast Communication. “agentMom shall support the ability to send unicast message” (3.2.1.1) and “to receive unicast message” (3.2.1.2). “Unicast message shall only be received by the specified address” (3.2.1.3) and “shall arrive at the specified address and in order” (3.2.1.4).	Entity: the unicast message. Sending and receiving are its lifecycle; addressing and ordering are the rules that govern it: one FR under the assignment’s grouping rule. 3.2.1.1 and 3.2.1.2 are Driving Requirements. Risk: TCP keeps messages in order only *within a single connection*. The SRS never says when a connection opens or closes, so the ordering of messages after a reconnection is undefined.	A1.1 Unicast messages travel over TCP. A1.2 One connection is kept open for each sender-to-receiver pair, so TCP’s own ordering is enough and messages are not re-ordered again by the application. The sequence number is used only for checking.	A1.1: Supported by SRS (2.1.2 states TCP/IP is used for unicast). A1.2: Design decision. The SRS says nothing about when connections open or close; a reconnect would break the ordering, so keeping one connection open is our choice, not the SRS’s.
3.2.2.3 – 3.2.2.6, 3.2.2.9	FR	Multicast group membership. “agentMom shall support the ability to send request to join multicast group” (3.2.2.3) and “to leave multicast group” (3.2.2.4). “agentMom shall not allow receiving multicast message from a group before joining” (3.2.2.5) “or after leaving that multicast group” (3.2.2.6). “agentMom shall support the ability to receive multicast message from multiple groups” (3.2.2.9).	Entity: the agent’s membership state in a group. 3.2.2.3 and 3.2.2.4 are Driving Requirements. Not CRUD: what matters is the *delivery decision that membership drives* (3.2.2.5/3.2.2.6), not the storing of a membership record. 3.2.2.9 is grouped here rather than with messaging because receiving from several groups at once is a property of membership; it cannot be tested apart from joining. Risk: a leave request and an incoming message arriving at the same moment.	A2.1 Leaving takes effect at once: the membership list is updated first, before the operating system is asked to drop the group, so a message already on its way is dropped rather than delivered. A2.2 When an agent belongs to several groups, each joined group is given its own socket so every message can be matched to the right group.	A2.1: Design decision. The SRS does not describe what happens to messages already in transit during a leave; this behaviour was decided by us. A2.2: Design decision. The SRS specifies neither the socket strategy nor how a message is matched to a group.
3.2.2.1, 3.2.2.2, 3.2.2.7, 3.2.2.8	FR	Multicast messaging. “agentMom shall support the ability to send multicast message” (3.2.2.1), “to receive multicast message” (3.2.2.2), “to set time-to-live for multicast message” (3.2.2.7), and “to set multicast address and port for sending and receiving multicast message” (3.2.2.8).	Entity: the multicast message. Sending and receiving are its lifecycle; the time-to-live (TTL) and the destination address and port are its attributes, set when the message is sent: the same structure that makes create/view/edit/delete of one record a single FR. 3.2.2.1 and 3.2.2.2 are Driving Requirements.	A3.1 A default TTL applies when the sender does not set one. A3.2 The TTL is also checked by the receiving application (a TTL of zero or less means the message is not delivered). This is a different mechanism from the router-hop definition in §1.3. A3.3 Messages above a fixed size limit are refused when sent, rather than	A3.1: Unsupported. The SRS requires the ability to set a TTL but is silent on a default, so this value was introduced during development. A3.2: Design decision, explicitly noted as diverging from §1.3. It exists only because the SRS mechanism cannot be observed on one machine.

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
			Risk: SRS §1.3 defines TTL as a count of router hops. On a single-machine test bed no router is crossed, so TTL has no visible effect on the network and cannot be demonstrated the way the SRS defines it.	being split into pieces.	A3.3: Design decision. An implementation safety rail, not taken from the SRS.
3.2.3.1, 3.2.3.2, 3.2.3.3 †	FR	Broadcast communication. “agentMom shall support the ability to sent [sic] broadcast message” (3.2.3.1) and “to receive broadcast message” (3.2.3.2). “Broadcast message shall be sent to all possible hosts under the same local network” (3.2.3.3).	All three sub-requirements are Driving Requirements. Wording: the SRS says a message shall be *sent to* all hosts, not that it shall reach them. This is consistent with 2.4.1, which permits delivery to none. We assert the send, not the delivery. Risk: constraint 2.4.4 warns broadcast may be administrator-only. On a single-machine test bed “all possible hosts” cannot be listed, so 3.2.3.3 cannot be checked at network scale.	A4.1 The limited broadcast address 255.255.255.255 satisfies “all possible hosts under the same local network”, with a subnet-directed address used instead where the operating system will not send to it. A4.2 The development and test machines allow broadcast traffic without administrator permission.	A4.1: Design decision. The SRS names no broadcast address. A4.2: Unsupported. Constraint 2.4.4 states the opposite may hold, so this is a gap on our side rather than something the SRS supports.
3.2.4.1 – 3.2.4.4	FR	Unicast encryption. “agentMom shall support the ability to encrypt unicast message” (3.2.4.1) and “to decrypt unicast message” (3.2.4.2). “agentMom shall allow an agent to decide whether or not to encrypt a message” (3.2.4.3). “agentMom shall automatically decrypt encrypted message” (3.2.4.4).	Entity: the security state of a unicast message: applying it, removing it, and who decides. 3.2.4.1 and 3.2.4.2 are Driving Requirements. Risk: 3.2.4.3 leaves the unencrypted path available by choice at any time, so confidentiality is a property of each message rather than of the system as a whole.	A5.1 AES-256-GCM is the encryption used. A5.2 Each pair of agents shares one key, worked out in advance from the pair’s identity and a project seed rather than exchanged between them; there is no key-exchange handshake. A5.3 A message marked as encrypted but missing its IV or authentication tag is treated as malformed and dropped, with no attempt made to decrypt it.	A5.1: Design decision. The SRS names no algorithm (see NFR10). A5.2: Unsupported. Section 2.5.3 describes a key holder for the multicast case only and says nothing about unicast keys, so this was added during development. A5.3: Design decision. The SRS does not say how malformed messages should be handled.
3.2.4.5, 3.2.4.6	FR	Multicast encryption. “agentMom shall support the ability to encrypt multicast message” (3.2.4.5) and “to decrypt multicast message” (3.2.4.6).	Extends encryption to group communication, which carries more weight than the unicast case because one leaked group key exposes every member at once. Risk: getting the shared key to members, and the absence of any way to withdraw it as membership changes.	A6.1 A key-holder agent exists and issues the group key only to agents on its allow-list. A6.2 The group key is never changed or withdrawn when a member leaves, so a departed agent keeps the ability to decrypt group traffic. A6.3 Key requests and key responses are always encrypted, overriding the per-message opt-out in 3.2.4.3.	A6.1: Supported by SRS (2.5.3 states a key-holder agent maintains a list of agents allowed to get the keys). A6.2: Unsupported. The SRS is silent on changing or withdrawing keys; this simplification is ours. A6.3: Design decision. Sending a key in plain text would defeat

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
					the mechanism, but the SRS does not require this.
3.2.5.1, 3.2.5.2	FR	Conversation control architecture. “agentMom [with] shall support the use of the architecture that agent directly controls the conversations” (3.2.5.1) and “the architecture that agent’s components control the conversations” (3.2.5.2).	Both variants are Driving Requirements and represent a real architectural decision rather than routine behaviour. Risk: the two control styles may need different internal routing, which would duplicate a large part of the implementation.	A7.1 Both architectures sit behind one shared handler interface over the same transport layer; only the conversation control logic differs. A7.2 The architecture can be switched while the system is running, without restarting the agent’s sockets.	A7.1: Design decision. The SRS requires both architectures but does not say whether they share a transport layer; sharing was our choice, to avoid duplication. A7.2: Design decision. Added so the switch can be demonstrated; the SRS does not require switching while running.
3.2.6.1	NFR Compatibility (ISO/IEC 25010: Compatibility / Co-existence)	Compatibility. “The new built agentMom shall be compatible with the agentMom 1.2” (3.2.6.1).	Classified as an NFR because it constrains a quality property of the product rather than specifying a function it performs, despite appearing inside §3.2. It matters because any existing agentMom 1.2 deployment should keep working once this project is added on top of it. Risk: no agentMom 1.2 artifact is available to us, and our implementation does not run on the Java version the SRS specifies (2.1, 2.1.1). Compatibility therefore cannot be checked against the real 1.2.	A8.1 “Compatible” is read as: the existing 1.2 public method signatures are left unchanged, and new features are added alongside them rather than by changing existing methods. A8.2 The 1.2 interface is rebuilt by us from SRS §2.2, because no reference implementation can be obtained.	A8.1: Design decision. The SRS requires compatibility but never defines it at the interface level, so this reading is ours. A8.2: Unsupported. A contract we wrote ourselves cannot evidence compatibility with the real 1.2. Carried forward as a declared limitation in Part 3.
2.4.1	NFR Reliability (ISO/IEC 25010: Reliability / Fault tolerance)	Reliable message delivery. “multicast/broadcast packets are delivered with best effort. Thus, a packet may be delivered to all specified agents or none” (2.4.1).	Sets the delivery expectation for two of the three transports and directly shapes how delivery can be tested. Taken from §2.4 Constraints rather than the numbered requirements in §3.2; treated as an NFR because it constrains a quality property rather than describing a function. Risk: as worded the requirement cannot be proved wrong: every outcome that can be observed satisfies it: and the SRS states no	A9.1 Best-effort delivery permits loss. A9.2 No acknowledgement, retry or resend layer is therefore built on the multicast or broadcast paths.	A9.1: Supported by SRS (2.4.1 states this directly). A9.2: Design decision. 2.4.1 permits failure but does not forbid working around it; the decision not to work around it is ours, made so that observed behaviour stays faithful to the constraint.

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
			delivery-rate threshold. We also note that real UDP can deliver to some recipients, which the SRS's all-or-none wording does not admit; that divergence is our own technical observation, not SRS text.		
2.4.2	NFR Security (ISO/IEC 25010: Security / Confidentiality)	Security. "we provide some basic mechanisms for security such as message encryption. However, there is no guarantee that the others cannot decrypt the encrypted messages" (2.4.2).	Sets the real security bar for the system, which is lower than the phrase "secured communication" in the project title suggests, and bounds what can honestly be claimed in the final judgment. Also taken from §2.4 Constraints, for the same reason as 2.4.1. Risk: as worded nothing is promised, so nothing can be proved wrong. No algorithm, key length or strength target is named anywhere in the SRS.	A10.1 A standard symmetric algorithm is acceptable for this project; AES-256-GCM was chosen. A10.2 No claim about strength, key length or resistance to attack is made or tested anywhere in the implementation or the report.	A10.1: Design decision. The SRS names no algorithm or key length, so the choice is ours and is not backed by the SRS. A10.2: Supported by SRS (2.4.2 explicitly disclaims any guarantee, so making no strength claim follows directly from the SRS).

Notes supporting the table above

Two of the three NFRs (2.4.1, 2.4.2) come from the SRS's Constraints section rather than §3.2; the third (3.2.6.1) is reclassified here as non-functional, each anchored to an ISO/IEC 25010 characteristic rather than our own assertion. Two further SRS clauses outside §3.2 bear on this scope: product function 2.2.4 is satisfied compositionally by FR1, FR3, and FR4 plus the harness's mode selection, and constraint 2.4.3 (router/NIC/OS multicast support) is a hard precondition for every multicast test, so cases are blocked rather than failed where it's unmet. The GUI itself is not an SRS requirement: the SRS describes a Java developer framework with no interface of its own, so the harness exists solely to satisfy this assignment's observability requirement, and no expected result in Part 3 is asserted about the harness itself, only about framework behaviour observed through it. Requirement wording is quoted verbatim wherever exact phrasing carries weight, since paraphrasing 3.2.3.3 ("sent to," not "reach") or 2.4.1 ("all specified agents or none") would make the corresponding tests indefensible; a printed typo in 3.2.3.3 ("3.3.3.3" in SRS v1.1) is cited as 3.2.3.3 with the discrepancy recorded, not silently corrected. Section 3.1's four use cases (join/leave a multicast group; send/receive unicast, multicast, broadcast) are the basis for Part 3's manual system-level tests, since their expected results come directly from the SRS rather than our interpretation of it.

Note 6: Testability of the three NFRs, declared in advance

None of the three selected NFRs can be tested as a simple pass/fail, because of how the SRS itself has worded them: that's a gap in the specification, not something we got wrong. Where the SRS gives no clear threshold to test against, we say so instead of inventing one. We are stating this here, before collecting any evidence, and for each NFR we test a related, checkable stand-in instead, with the gap between the requirement and the stand-in spelled out clearly below.

NFR	Why it can't be tested directly	What we'll test instead	Limitation to be declared in Part 3
NFR 3.2.6.1 Compatibility	Requires compatibility with an artifact we do not have, in a language we are not using (2.1, 2.1.1).	The rebuilt 1.2 method signatures are unchanged by any new feature, and a call path that uses only the older methods still works while every new feature is switched on.	No genuine 1.2 artifact exists and the 1.2 interface is our own reconstruction (A8.2), so a contract we wrote ourselves cannot fail. This evaluates internal additive discipline, not real compatibility.
NFR 2.4.1 Reliability	Delivery to "all specified agents or none" is satisfied by every possible observation, so no test can fail.	No acknowledgement, retry or resend path exists anywhere on the multicast or broadcast paths, and a dropped message is never sent again.	The SRS gives no delivery-rate threshold, so no pass/fail rate can be claimed. What we check instead is that no retry code exists, by inspecting the source and observing that a dropped message is never sent again.
NFR 2.4.2 Security	The clause explicitly disclaims any guarantee, so nothing is promised and nothing can be checked against it.	What travels over the network differs from the original readable message, and decrypting with a wrong or tampered key fails outright rather than producing partly readable text.	No claim about strength, key length or resistance is made or tested (A10.2). Any SonarQube security hotspot on our key derivation is expected and corroborates the declared A5.2 gap.

This reliability check is unusual because it looks for the absence of code rather than a behaviour: assumption A9.2 is satisfied simply by there being no retry path at all. The evidence is a code search that finds nothing, plus an observed dropped message that is never resent.

Part 2: AI-Generated GUI Baseline

The GUI is a small React front-end (packages/web) built to drive the underlying framework and show real success/error feedback for each selected requirement: it's a test harness, not a polished product interface. (Assumption A11: the SRS describes a developer framework with no interface requirements of its own, so the GUI exists only to make the framework's behaviour observable for this assignment.) It has one page per functional area: Unicast, Multicast, Broadcast, Security (encryption for both unicast and multicast), Compatibility, Architecture (the conversation-control switch), Admin (managing the demo agents and groups), and a Dashboard with a live event log. Each page runs real actions against four actual agent processes on the same machine, and shows whatever genuinely happens: a message delivered or dropped, a permission denied, a TTL expiry: rather than a simulated or mocked result.

All ten requirements can be exercised through the GUI: FR1–FR4 on their own pages, FR5–FR6 on the Security page, FR7 through the Architecture page's switch, and NFR8 on the Compatibility page. NFR9 and NFR10 aren't things you can click through on screen: they're checked by inspecting the code instead, as explained in Part 3. A full requirement-to-page-to-test mapping is kept in FEATURE-MAP.md.

The baseline was frozen before any SonarQube or testing evidence was collected, at commit e79b197 (recorded in evidence/BASELINE_TAG_HASH.txt). The SonarQube scan in Part 3.A was run one commit later (tag baseline-v1-sonar, 72aedfb): that commit only records the scan evidence and adds no application code. No source file under packages/ changed between the two commits.

Full setup and run instructions are in README.md and oneliner how to run.txt, so they aren't repeated here. In short: npm install (Node.js ≥ 20), npm run build, then bash runsystem.sh: one script that starts the control plane, the four demo agents, and the web GUI at http://localhost:5173. Ctrl-C shuts everything down cleanly.

The implementation was built using AI-assisted coding, following a human-written plan (docs/agentmom-implementation-plan-v2.md) based on the SRS. Two deviations from that plan happened during development and are recorded in DEVIATIONS.md: we used Node.js/TypeScript instead of the SRS's specified Java (tied to the Compatibility limitation, A8.2), and we changed the front-end framework choice to keep the SonarQube scan focused on code we actually wrote. All 24 AI-introduced assumptions are logged in ASSUMPTIONS.md, each labelled supported-by-SRS, design-decision, or unsupported: that file is the source Part 1's table is built from, and a test (dod.test.ts) checks it stays in sync with the code.

Part 3: Quality Evaluation

A. SonarQube Report and NFR Evaluation

SonarQube Community Build (v26.9.0.129388) was run locally via Docker against the complete frozen baseline: tag baseline-v1-sonar, commit 72aedfb8781e4304a88c0f01f2af3df5e7e5661b (the underlying application code was frozen one commit earlier, at e79b197, per evidence/BASELINE_TAG_HASH.txt). The scan used sonar-scanner CLI 6.2.1.4610, run from the repository root. sonar-project.properties scopes the analysis to all four workspace packages' src and tests in full (packages/core, packages/agent, packages/control-plane, packages/web); no file or package was hand-picked or omitted, and only generated/vendor paths (node_modules, dist, *.d.ts) are excluded. Coverage was measured with npm run coverage (vitest + v8, 217 tests) and supplied to the scan through coverage/lcov.info. The analysis task completed with status SUCCESS in 9.8 seconds. Raw exports (sonar-measures.json, sonar-issues-full.json, sonar-hotspots.json) are retained in evidence/sonarqube/, alongside dashboard screenshots.

Reported metrics: 3,289 lines of code; Quality Gate passed; Security rating B (19 open issues, all Low severity); Reliability rating D (14 open issues: 1 High, 1 Medium, 12 Low); Maintainability rating A (50 open issues: 18 Medium, 32 Low); 88.8% coverage; 0% duplication; and 0 Security Hotspots (confirmed by checking the hotspots list directly). 19 issues do sit under the "Security" area, but all 19 turn out to be the same low-severity code-smell pattern (see Finding F3 below), not real vulnerabilities, so they don't pull the rating down. It's also worth flagging that SonarQube Community Build doesn't scan for things like SQL injection or XSS, so this Security rating only covers what this edition checks for: it isn't a full security audit.

Selected findings

The five findings below were picked from the full list of 71 issues because each is either the most serious issue in its area, or representative of a pattern repeated across several files. Each is explained in terms of what it actually means for this codebase, not just counted.

Finding / quality area	What SonarQube reported & where	Why it matters	Action the evidence supports
F1: Sort with no comparison function in the crypto module (Reliability: Bug, Critical, rule S2871)	packages/core/src/crypto/symmetric.ts:59: “Provide a compare function that depends on String.localeCompare, to reliably sort elements alphabetically.”	This is the single Critical-severity Bug behind the project’s Reliability rating of D. A sort called with no comparison function falls back to comparing values as text, which is not guaranteed to stay stable or to	Confirm whether this sorted output feeds a hash or signature elsewhere in symmetric.ts; if so, replace the implicit sort with an explicit fixed-order comparison before the baseline is extended

Finding / quality area	What SonarQube reported & where	Why it matters	Action the evidence supports
		behave the same way on every machine. Where the sorted result is fed into a hash or signature, a different order changes the bytes that get hashed: a result that depends on the environment, not just untidy code.	further: this affects correctness, not just style.
F2: Missing table header markup on the Admin page (Reliability: Bug, Major, rule S5256 (accessibility, wcag2.1-a))	packages/web/src/pages/Admin.tsx:47: the table has no header row or column attached to it.	Screen readers cannot tell which column a data cell belongs to, so the admin view is not usable with assistive technology. This is directly relevant to any usability or accessibility claim made about the harness.	Add proper header cells (<th scope="col">) to the table. Use as supporting evidence if usability is later scoped as a formal NFR, or as a defect entry if manual testing confirms the same gap.
F3: Multicast group address written out by hand in four files (Security: Code Smell, Low × 19, rule S1313 (“hardcoded IP address”))	239.1.1.5 / 239.1.1.6 are typed out directly in packages/agent/src/config.ts, packages/control-plane/src/agent-registry.ts, packages/web/src/pages/types.ts and packages/web/src/components/AgentTopologyGraph.tsx: all 19 of the project’s Security-area issues are this one pattern.	Individually each flag is a justified false positive: 239.x.x.x is a standard multicast address, required by FR3/FR4 and consistent with assumption A4.1, not a leaked or unsafe endpoint. But repeating it across four files is a genuine maintainability risk: the same value is retyped instead of imported, so changing the group address later takes four edits that all have to agree.	No security action needed (reviewed and justified). Maintainability action: move both addresses into packages/core/src/constants.ts and import them everywhere else, which would also clear all 19 Security-area flags at the source.
F4: Unclear spacing between labels and inputs, repeated on every harness page (Reliability / Maintainability: Code Smell, Major × 12, rule S6772 (“Ambiguous spacing”))	Broadcast.tsx, Compatibility.tsx, Multicast.tsx, Security.tsx, Unicast.tsx and EncryptionToggle.tsx each flag a label or piece of text sitting directly against a dropdown, input or code element with no explicit space between them.	This is 12 of the 14 Reliability-area issues (86%) and the largest single pattern in the codebase. It can produce inconsistent label spacing in the running GUI: something a user would see, not just a lint preference.	Check on each affected page whether the label reads correctly (e.g. “TTL:5” vs “TTL: 5”). This is a natural candidate to fold into one of the required manual system-level test cases, and should be logged as a defect only if the spacing on screen is actually wrong: a Sonar flag alone is not a confirmed defect (Part 4).
F5: Component inputs not marked read-only, across the whole web package (Maintainability: Code Smell, Minor × 14, rule S6759)	Nearly every component under packages/web/src/components and pages/*.tsx (for example StatusBadge.tsx, ArchitectureSwitch.tsx, EncryptionToggle.tsx, MessageComposer.tsx, ReliabilityDial.tsx) declares the values it receives without marking them read-only.	With 14 occurrences this is the second-largest pattern after F4 and a meaningful contributor to the 50 open Maintainability issues. It is a style and consistency gap from how components were written during AI-assisted development, not a functional defect: no observed bug traces back to it.	Adopt read-only component inputs as the standard signature going forward. Low priority: it does not block the Quality Gate and is not a Jira candidate, since no repeatable failure in behaviour is associated with it.

NFR evaluation

The three NFRs selected in Part 1 (3.2.6.1 Compatibility, 2.4.1 Reliability, 2.4.2 Security) were already flagged in Note 6 as not directly testable in the SRS's own wording. The table below evaluates each against the stand-in test described there, using SonarQube evidence only where it's actually relevant, and saying plainly where it isn't.

NFR	SonarQube evidence (if relevant)	Other evaluation method	Finding / judgment	Limitation
NFR 3.2.6.1	N/A: static analysis cannot	Manual inspection of the	This check passes: no older	Carried forward from A8.2:

NFR	SonarQube evidence (if relevant)	Other evaluation method	Finding / judgment	Limitation
Compatibility	compare our code against an external agentMom 1.2 artifact. The only tangential finding is two Minor naming-convention smells (rule S101) on packages/agent/src/legacy /agentmom-1_2-adapter.ts (the AgentMom1_2 interface and class names), which are cosmetic and do not affect the preserved signatures.	legacy adapter's exported method signatures against the rebuilt 1.2 interface (A8.2), confirming no existing method was altered and new features sit alongside it.	method signature was changed: but that only shows internal consistency against our own reconstruction of the interface, not real compatibility with the actual agentMom 1.2.	a 1.2 contract we wrote ourselves cannot evidence real compatibility. This is unchanged and unresolved by the SonarQube run.
NFR 2.4.1 Reliability (best-effort delivery)	Not directly relevant: SonarQube's "Reliability" rating (D) measures bugs in the code (led by F1, the crypto sort issue), which is a different thing from the SRS's delivery-reliability constraint; the two share a name but should not be conflated. No retry- or acknowledgement-related code smell was raised in the transport files, which is at least consistent with assumption A9.2.	Search of packages/agent/src/transports and packages/control-plane/src for any retry, acknowledgement or resend logic, plus what the FR3/FR4 integration tests show when a message is dropped.	No acknowledgement or resend path exists anywhere in the multicast or broadcast code; this matches assumption A9.2 exactly, so the implementation is faithful to the "may deliver to all or none" wording.	The SRS gives no delivery-rate threshold, so no pass/fail rate can be claimed here either: this check can only confirm that no retry mechanism exists, not measure actual delivery reliability under load.
NFR 2.4.2 Security	Relevant, but narrow: Security Hotspots: 0 (confirmed via api/hotspots/search), Security rating B with all 19 issues being the single justified multicast-address pattern (F3); none relate to the encryption or key-derivation code. Community Build's own banner discloses that it does not scan for deeper vulnerability classes, and it is not designed to catch a weakness in the design of a protocol, such as a key worked out in advance with no handshake.	Manual code inspection of packages/core/src/crypto/symmetric.ts and the key-holder logic against assumptions A5.2/A6.2 (shared key worked out in advance, and no change of key when a member leaves).	The absence of a SonarQube hotspot on the key-derivation code must not be read as a security clearance; manual inspection confirms the pre-derived-key and no-key-change gaps declared in Part 1 are real and present in the frozen baseline.	This directly corroborates SRS 2.4.2's own disclaimer ("no guarantee that others cannot decrypt the encrypted messages"): the tool's silence here is a limitation of the tool, not evidence of security.

Evidence retained: full JSON export of all 71 issues, the empty hotspots result, and the summary measures export (api/issues/search, api/hotspots/search), plus dashboard screenshots, are kept in evidence/sonarqube/ alongside this report.

B. Functional Test Derivation, Execution and Traceability

All 15 test cases were executed against the frozen baseline (commit e79b197); no file under packages/ was changed to produce this evidence. Full raw evidence is in evidence/test-execution/.

Table A: Test Condition Record

Conditions derived from all 7 FRs are recorded in full in TEST-CONDITIONS.md (54 total); the 24 rows below back the 15 executed test cases.

Test basis / requirement	Condition ID	Test condition
3.2.1.1/.2	COND-04	A send opens a connection and the addressed agent receives the message
BR-01	COND-05	A message addressed to a different agent is dropped and raises PROTOCOL_VIOLATION
3.2.1.3	COND-06	A message sent A→B is not received by C
3.2.1.4	COND-07	50 messages sent A→B arrive in the order they were sent
BR-03	COND-08	A sequence number two ahead of the last one seen raises SEQUENCE_ANOMALY; the next one in order does not
SRS UC2	COND-10	Use Case 2 (unicast) re-enacted end-to-end through the harness
3.2.2.3	COND-11	Joining moves the agent from NOT_MEMBER to JOINING to MEMBER
BR-06	COND-13	Leaving removes the agent from the membership list before the operating system is asked to drop the group
3.2.2.9 / BR-07	COND-16	An agent joined to α and β receives from both, with each message matched to the right group
SRS UC1	COND-17	Use Case 1 (join/leave) re-enacted through the harness
BR-06	COND-18	A message arriving at the same moment as a leave is dropped, not delivered
BR-09	COND-19	A TTL of 1 is deliverable; a TTL of 0 is expired: the exact boundary
BR-09	COND-21	A received message whose TTL has expired raises MESSAGE_DROPPED_TTL_EXPIRED and is not delivered
SRS §1.3	COND-22	Setting the multicast TTL to 0 at operating-system level keeps a message on the machine that sent it
BR-10	COND-24	A destination outside the multicast range 224.0.0.0/4 is rejected when the send is attempted
3.2.3.1/.2	COND-29	A broadcast send is received by every agent on the same host
3.2.3.3	COND-30	A broadcast is sent to all possible hosts under the same local network
SRS UC4	COND-33	Use Case 4 (broadcast) re-enacted through the harness
BR-16	COND-37	A message marked as encrypted but missing its IV or authentication tag is malformed: dropped, with no attempt to decrypt
BR-14	COND-38	A message sent unencrypted travels as

Test basis / requirement	Condition ID	Test condition
		readable text; the same message sent encrypted looks different on the wire
3.2.4.4	COND-39	The receiver decrypts automatically, with no action from the user
BR-18	COND-42	A key request from an agent not on the allow-list is denied and raises GROUP_KEY_DENIED
3.2.5.1/.2	COND-47	Delivery behaves the same before and after a live architecture switch
3.2.5.1/.2	COND-49	The switch is visible in the harness through an ARCHITECTURE_SWITCHED marker

Table B: Test Case Record

Executed 2026-09-07 against the frozen baseline (e79b197), single-host test bed. Automated cases: npm test, log in evidence/test-execution/npm-test-output-verbose.txt (217/217 green). Manual cases: driven live through the harness (bash runsystem.sh), screenshots in evidence/test-execution/. Category minimums met: boundary (TC-03, TC-05, TC-07 - 3 of 2 required), invalid/error (TC-09, TC-12, TC-14 - 3 of 2), manual system-level (TC-01, TC-04, TC-10, TC-11, TC-15 - 5 of 3), genuine FAILED/BLOCKED (TC-08, TC-11 - 2 of 2).

TC-01: Unicast delivery, SRS Use Case 2

ID / title	TC-01: Unicast delivery, SRS Use Case 2
Level / category	System: manual, Normal
Test basis / objective	COND-10 (SRS UC2) + COND-04 (3.2.1.1/.2): a unicast send is established and delivered to the addressed agent
Preconditions	Demo topology running (agent-A…D started, control plane up)
Test data	3 messages, agent-A → agent-B, unencrypted
Steps	Open the Unicast page; set from=agent-A, to=agent-B; send 3 messages one after another; watch the status badge and the log after each
Expected result	Each send is received by agent-B, in the order sent (3.2.1.4 applies transitively)
Actual result	All 3 delivered; log timestamps strictly increasing (SENT then RECEIVED within 3ms, each pair complete before the next SENT)
Status	PASSED
Evidence	TC-01-unicast-burst-success.png

TC-02: Unicast addressing isolation

ID / title	TC-02: Unicast addressing isolation
Level / category	Integration: automated, Business rule
Test basis / objective	COND-06 (3.2.1.3) + COND-05 (BR-01): a message addressed to B must not be seen by C
Preconditions	Demo topology started inside the test (fr1-unicast.test.ts)
Test data	One message, agent-A → agent-B
Steps	Send a unicast A→B through the control plane (POST /messages/unicast); read the combined log; check that every receive event for this exchange belongs to agent-B only

Expected result	No receive event recorded against agent-C
Actual result	Matched: only agent-B logged as receiver
Status	PASSED
Evidence	fr1-unicast.test.ts > TC-02 / COND-06 … > only B receives: npm-test-output-verbose.txt

TC-03: Ordered delivery of a 50-message burst

ID / title	TC-03: Ordered delivery of a 50-message burst
Level / category	Integration: automated, Boundary
Test basis / objective	COND-07 (3.2.1.4) + COND-08 (BR-03): TCP-backed ordering holds under volume, with no false SEQUENCE_ANOMALY
Preconditions	As TC-02
Test data	50 messages in sequence, agent-A → agent-B
Steps	Send 50 messages in a loop; check that no SEQUENCE_ANOMALY event is raised; check that at least 50 send events are recorded for agent-A
Expected result	No anomaly raised; all 50 sent
Actual result	Matched
Status	PASSED
Evidence	fr1-unicast.test.ts > TC-03 / COND-07 … > ordered delivery: npm-test-output-verbose.txt

TC-04: Multicast join/leave, SRS Use Case 1

ID / title	TC-04: Multicast join/leave, SRS Use Case 1
Level / category	System: manual, Normal
Test basis / objective	COND-17 (SRS UC1) + COND-11 (3.2.2.3): joining changes membership and unlocks delivery; leaving takes it away
Preconditions	agent-A starts as NOT_MEMBER of group α (239.1.1.5)
Test data	Group α, sender agent-C
Steps	(1) confirm agent-A is NOT_MEMBER; (2) join agent-A; (3) send a multicast from agent-C; (4) confirm agent-A receives it; (5) have agent-A leave; (6) send again from agent-C; (7) confirm agent-A does not receive it
Expected result	agent-A receives only while a member (3.2.2.5, 3.2.2.6)
Actual result	After join: message 4c3289fb… received by A, B, C and D. After leave: message 9327f9fa… received by B, C and D only, with agent-A absent
Status	PASSED
Evidence	TC-04-multicast-joined.png, TC-04-multicast-received-after-join.png, TC-04-multicast-not-received-after-leave.png

TC-05: Leave/inject race (BR-06 boundary)

ID / title	TC-05: Leave/inject race (BR-06 boundary)
Level / category	Integration: automated, Boundary
Test basis / objective	COND-18 (BR-06) + COND-13: a message arriving at the same moment as a leave must be dropped, because the membership list is cleared first, before the operating system is asked to drop

	the group
Preconditions	agent-B is a member of α
Test data	agent-B leaves α while agent-C sends a multicast at the same moment
Steps	Trigger the combined leave-and-send endpoint (POST /groups/239.1.1.5/leave-then-inject) with agentId=agent-B and injectFrom=agent-C; check the log for any receive event recorded against agent-B
Expected result	No receive event for agent-B on that message
Actual result	Matched: zero
Status	PASSED
Evidence	fr2-6-7.test.ts > TC-05 / COND-18 … > leave-then-inject drops the in-flight datagram: npm-test-output-verbose.txt

TC-06: Multi-group attribution

ID / title	TC-06: Multi-group attribution
Level / category	Integration: automated, Normal
Test basis / objective	COND-16 (3.2.2.9 / BR-07): an agent in two groups matches each message to the group it came from
Preconditions	agent-C joined to both α and β
Test data	One send to α, one to β
Steps	Send a multicast to α, then to β; check which group each received message is recorded against for agent-C
Expected result	The α send recorded against 239.1.1.5 and the β send against 239.1.1.6, with no mix-up between them
Actual result	Matched
Status	PASSED
Evidence	fr2-fr3-delivery.test.ts > COND-26 / TC-06 area …: npm-test-output-verbose.txt

TC-07: TTL expiry boundary

ID / title	TC-07: TTL expiry boundary
Level / category	Integration: automated, Boundary
Test basis / objective	COND-19 (BR-09, exact boundary of TTL 1 against TTL 0) + COND-21: a message whose TTL has expired is dropped by the receiving application
Preconditions	None beyond demo topology
Test data	Multicast with ttl=0
Steps	Send a multicast with ttl=0; check the log for MESSAGE_DROPPED_TTL_EXPIRED
Expected result	Message dropped, event raised, no receive event for it
Actual result	Matched
Status	PASSED
Evidence	fr3-ttl.test.ts > TC-07 / COND-21 … > ttl=0 at the application layer is dropped: npm-test-output-verbose.txt

TC-08: OS-level TTL confinement (SRS §1.3): deliberately expected FAILED

ID / title	TC-08: OS-level TTL confinement (SRS §1.3): deliberately expected FAILED
Level / category	Integration: automated, Normal
Test basis / objective	COND-22: SRS §1.3 defines TTL as a count of router hops; setting the multicast TTL to 0 should keep a message on the machine that sent it
Preconditions	Single-host test bed (no router in the path)
Test data	Multicast with a positive TTL inside the message, so only the operating-system TTL could hold it back
Steps	Send a multicast to group α; record which agents on the same machine receive it
Expected result	(Per SRS §1.3) the message does not leave the machine that sent it
Actual result	agent-C, agent-B and agent-D all received it: delivery on the same machine crosses no router, so hop-count confinement has nothing to act on
Status	FAILED (against the SRS expectation: see Part 4 §5.1 for the Vitest-green-vs-SRS-FAILED distinction)
Evidence	fr3-ttl.test.ts > TC-08 / COND-22 … console line: '[TC-08] co-located receivers observed … agent-C, agent-B, agent-D': npm-test-output-verbose.txt

TC-09: Multicast address range validation

ID / title	TC-09: Multicast address range validation
Level / category	Integration: automated, Invalid/error
Test basis / objective	COND-24 (BR-10): a destination outside the multicast range 224.0.0.0/4 is rejected when the send is attempted
Preconditions	None
Test data	groupAddress 10.0.0.1
Steps	Attempt a multicast send to groupAddress=10.0.0.1 (POST /messages/multicast)
Expected result	Rejected, HTTP 400
Actual result	Matched
Status	PASSED
Evidence	fr3-ttl.test.ts > TC-09 / COND-24 … > the control plane returns an error for 10.0.0.1: npm-test-output-verbose.txt

TC-10: Broadcast delivery, SRS Use Case 4 (host-local)

ID / title	TC-10: Broadcast delivery, SRS Use Case 4 (host-local)
Level / category	System: manual, Normal
Test basis / objective	COND-29 (3.2.3.1/.2) + COND-33 (SRS UC4): one broadcast message is sent out to every host, observed here as every agent on this machine
Preconditions	Demo topology running
Test data	One broadcast from agent-A
Steps	Open the Broadcast page; set from=agent-A; send; watch the per-agent receipt indicators and the address that was used

Expected result	All 4 agents show received; the address used is reported
Actual result	agent-A, B, C and D all show ‘received’; address used = 255.255.255.255
Status	PASSED
Evidence	TC-10-broadcast-hostlocal-received.png

TC-11: LAN-wide broadcast reach, SRS 3.2.3.3: BLOCKED

ID / title	TC-11: LAN-wide broadcast reach, SRS 3.2.3.3: BLOCKED
Level / category	System: manual, Normal
Test basis / objective	COND-30: a broadcast must be sent to ‘all possible hosts under the same local network’
Preconditions	Required: a local network with more than one machine, so that ‘all possible hosts’ can be listed and receipt confirmed on each. Available: one physical machine only
Test data	Same broadcast as TC-10
Steps	Attempt to confirm receipt on a second machine on the same local network
Expected result	N/A: the precondition itself cannot be set up
Actual result	The required precondition (two or more machines on the same local network) does not exist on this test bed, confirmed by scripts/verify-preconditions.sh (one host, 5 interfaces, zero peers). This is not a claim that LAN-wide delivery failed: it was never run at LAN scale. TC-10 already establishes that host-local delivery works
Status	BLOCKED
Evidence	TC-11-single-host-network-interfaces.txt (verify-preconditions.sh output)

TC-12: Malformed encrypted envelope

ID / title	TC-12: Malformed encrypted envelope
Level / category	Unit: automated, Invalid/error
Test basis / objective	COND-37 (BR-16): a message marked as encrypted but missing its IV or authentication tag must be dropped, with no attempt to decrypt
Preconditions	None
Test data	A message marked encrypted with the IV present and the authentication tag missing (and the opposite case, carrying fields that should not be there)
Steps	Run the envelope validator on the malformed message
Expected result	MALFORMED_CRYPTO_FIELDS (or STRAY_CRYPTO_FIELDS for the opposite case), with decryption never attempted
Actual result	Matched, on all 3 branches (missing authentication tag, stray IV, and a TTL set on a message that is not multicast)
Status	PASSED
Evidence	validator-and-sequence.test.ts > §8 invariants: BR-16 malformed crypto fields (TC-12 target): npm-test-output-verbose.txt

TC-13: Per-message encryption opt-in + auto-decrypt

ID / title	TC-13: Per-message encryption opt-in + auto-decrypt
Level / category	Integration: automated, Business rule
Test basis / objective	COND-38 (BR-14) + COND-39 (3.2.4.4): encryption is chosen by the sender for each message; the receiver decrypts automatically
Preconditions	None
Test data	One unicast sent with encryption on
Steps	Send an encrypted unicast A→B; confirm it is received with no decryption failure
Expected result	Delivered, encrypted on the wire, decrypted automatically, with no user action and no failure event
Actual result	Matched
Status	PASSED
Evidence	fr1-unicast.test.ts > TC-13 / COND-38+39 …: npm-test-output-verbose.txt; also shown live in TC-01-unicast-burst-success.png

TC-14: Key-request denial for a non-allow-listed agent

ID / title	TC-14: Key-request denial for a non-allow-listed agent
Level / category	Integration: automated, Invalid/error
Test basis / objective	COND-42 (BR-18): the key holder must refuse a request from an agent that is not on its allow-list
Preconditions	agent-B is a member of α but is not on agent-D's allow-list (allow-list and membership are deliberately separate)
Test data	Key request from agent-B for group α
Steps	Send a key request (POST /keys/request) with requestingAgentId=agent-B; check for GROUP_KEY_DENIED
Expected result	Denied
Actual result	Matched; the allow-listed control case (agent-C) is granted in the same run
Status	PASSED
Evidence	fr2-6-7.test.ts > TC-14 / COND-42 …: npm-test-output-verbose.txt

TC-15: Live conversation-architecture switch

ID / title	TC-15: Live conversation-architecture switch
Level / category	System: manual, Normal
Test basis / objective	COND-47 (3.2.5.1/.2, delivery identical on either side) + COND-49 (the switch is visible through a marker)
Preconditions	agent-A running agent-controlled
Test data	1 unicast A→B before the switch, 1 after
Steps	(1) send a unicast A→B while agent-controlled (delivered: TC-01); (2) switch agent-A to component-controlled; (3) confirm ARCHITECTURE_SWITCHED is logged; (4) send another unicast A→B; (5) confirm it is delivered
Expected result	Delivery identical on both sides; marker present; no socket restart
Actual result	ARCHITECTURE_SWITCHED logged for agent-A; the send after

	the switch (message c3b3b4c5…) was SENT and then RECEIVED 1ms later, the same timing as before the switch
Status	PASSED
Evidence	TC-15-architecture-switched.png, TC-15-delivery-after-switch.png; fr2-6-7.test.ts > TC-15 / COND-49 …: npm-test-output-verbose.txt

Table C: Traceability Record

Full 54-condition matrix. TC-nn = one of the 15 Table B cases above; other carriers are automated tests (all green, npm-test-output-verbose.txt) or a manual observation folded into an adjacent Table B case (noted explicitly).

Requirement	Condition	Test case	Execution result	Defect report
FR1: 3.2.1.1–.4	COND-01…03	framing.test.ts (unit)	PASSED	N/A
FR1	COND-04	unicast-transport.test.ts (component)	PASSED	N/A
FR1	COND-05	unicast-transport.test.ts (component)	PASSED	N/A
FR1	COND-06	TC-02	PASSED	N/A
FR1	COND-07	TC-03	PASSED	N/A
FR1	COND-08	validator-and-sequence.test.ts (unit)	PASSED	N/A
FR1	COND-09	unicast-transport.test.ts + fr1-unicast.test.ts	PASSED	N/A
FR1	COND-10	TC-01	PASSED	N/A
FR2: 3.2.2.3–.6, .9	COND-11…13	membership-and-multicast.test.ts (component)	PASSED	N/A
FR2	COND-14	fr2-6-7.test.ts (integration)	PASSED	N/A
FR2	COND-15	fr2-fr3-delivery.test.ts (integration)	PASSED	N/A
FR2	COND-16	TC-06	PASSED	N/A
FR2	COND-17	TC-04	PASSED	N/A
FR2	COND-18	TC-05	PASSED	N/A
FR3: 3.2.2.1, .2, .7, .8	COND-19, 25	framing.test.ts (unit)	PASSED	N/A
FR3	COND-20	membership-and-multicast.test.ts (component)	PASSED	N/A
FR3	COND-21	TC-07	PASSED	N/A
FR3	COND-22	TC-08	FAILED (expected: environment limitation)	N/A: see Part 4 §5.2
FR3	COND-23	security-and-config.test.ts (component)	PASSED	N/A
FR3	COND-24	TC-09	PASSED	N/A
FR3	COND-26	fr2-fr3-delivery.test.ts (integration)	PASSED	N/A
FR3	COND-27	observed during the TC-04 multicast walkthrough	PASSED (observed)	N/A

Requirement	Condition	Test case	Execution result	Defect report
FR4: 3.2.3.1–.3	COND-28, 31, 32	broadcast-keyholder-misc.test.ts (component)	PASSED	N/A
FR4	COND-29	TC-10	PASSED	N/A
FR4	COND-30	TC-11	BLOCKED (single-host precondition unmet)	N/A: see Part 4 §5.2
FR4	COND-33	folded into TC-10 (UC4 walkthrough)	PASSED	N/A
FR5: 3.2.4.1–.4	COND-34, 35, 36, 54	crypto.test.ts (unit)	PASSED	N/A
FR5	COND-37	TC-12	PASSED	N/A
FR5	COND-38, 39	TC-13	PASSED	N/A
FR5	COND-40	crypto.test.ts + security-and-config.test.ts	PASSED	N/A
FR6: 3.2.4.5, .6	COND-41, 42	TC-14	PASSED	N/A
FR6	COND-43, 44, 45	fr5-fr6-security.test.ts (integration)	PASSED	N/A
FR7: 3.2.5.1, .2	COND-46	broadcast-keyholder-misc.test.ts (component)	PASSED	N/A
FR7	COND-47, 48	fr7-architecture.test.ts (integration)	PASSED	N/A
FR7	COND-49	TC-15	PASSED	N/A
NFR8: 3.2.6.1	COND-50	broadcast-keyholder-misc.test.ts (component)	PASSED	N/A
NFR8	COND-51	manual: Compatibility page, live	PASSED	N/A
NFR9: 2.4.1	COND-52	traceability.test.ts (source inspection)	PASSED	N/A
NFR9	COND-53	fr2-fr3-delivery.test.ts (integration)	PASSED	N/A
NFR10: 2.4.2	COND-54	crypto.test.ts + fr5-fr6-security.test.ts	PASSED	N/A

52 of 54 conditions PASSED; 1 FAILED (COND-22 / TC-08, deliberate, environment-caused); 1 BLOCKED (COND-30 / TC-11, deliberate, precondition-caused). Zero produced a confirmed implementation defect; see the investigation in Part 4.

Part 4: Defect Reporting and Final Quality Judgment

5.1 The FAILED/BLOCKED-vs-defect distinction, applied to TC-08

Our test runner shows fr3-ttl.test.ts as passing, but that's only because the test checks which agents received the message: it doesn't check whether the SRS's TTL rule was actually followed. Judged against what the SRS itself says TTL should do (block delivery based on router hops), that didn't happen. So we're marking TC-08 as FAILED against the SRS's expectation, even though the test script itself didn't throw an error.

5.2 Investigation of every FAILED/BLOCKED case

TC-08: FAILED. We checked the setup, data and environment: one physical machine, no router in the network path. It reproduces every time (3 out of 3 runs this session, matching earlier CI runs too). Root cause: the SRS defines TTL as a router-hop counter, and since nothing on a single machine crosses a router, there's nothing for that mechanism to act on.

We confirmed the TTL call itself is correct by reading the code. Verdict: not a defect: already flagged in advance in DEVIATIONS.md (D-5). The implementation matches the SRS; our test setup just can't exercise the router-hop part of it.

TC-11: BLOCKED. We checked the precondition with a small script, which confirmed exactly one host with no other machine reachable on the network. There's nothing to reproduce here, since a BLOCKED test never runs. Root cause: the test needs more than one machine on the same network to check "all possible hosts," and we only have one. Verdict: not a defect: already flagged in advance in ASSUMPTIONS.md (A4.1/A4.2). TC-10 already shows the send/receive logic works at the scale we can actually test.

F1 (SonarQube, Part 3.A): symmetric.ts:59 implicit Array.sort(). We checked whether this sort feeds into a hash: it does: derivePairwiseKey() sorts the two agent IDs before hashing them. We then checked what JavaScript's default sort actually does with plain ASCII IDs: it sorts by character code, which is fixed and doesn't change between machines or locales (confirmed by a passing test, COND-36). SonarQube's suggested fix (localeCompare) would actually make the order depend on locale: the opposite of what's needed here. Verdict: not a defect. SonarQube is right that this pattern is risky in general, but in this specific case it happens to be safe.

F4 (SonarQube, Part 3.A): ambiguous JSX spacing, 12 occurrences. We checked whether the flagged labels actually render with a missing space, since SonarQube can't tell that from the code alone. We looked at every affected page while running TC-01, TC-04, TC-10 and TC-15: every label ("TTL", "port", "from", "to") displayed correctly, with proper spacing. Verdict: not a defect. It's a valid static-analysis flag, but nothing is actually wrong on screen.

5.3 Jira

Result of the triage: zero confirmed, reproducible implementation defects. Every FAILED or BLOCKED test, and every SonarQube finding that could plausibly point to a real bug (F1, F4), turned out to be "not a defect" for a specific, evidenced reason (see 5.2 above). As the assignment itself notes, not every failure or blocker should become a Jira bug: none of our four candidates met all four defect criteria (an executed test, an actual result that differs from expected, at least two reproductions, and no existing assumption already explaining it). So instead of opening Bugs, we logged each one in Jira as a closed, fully written-up investigation record, so the evidence trail exists in Jira itself, not just in this report.

Jira project: Vexa Testing (KAN), https://muhammadbilaltahir.atlassian.net/jira/software/projects/KAN. Epic KAN-9 ("SE3002 Assignment 01: Defect Triage") groups the four investigation records above: KAN-5 (TC-08), KAN-6 (TC-11), KAN-7 (SonarQube F1), KAN-8 (SonarQube F4): each marked Done / Low priority and containing the environment, preconditions, reproduction steps, expected vs. actual result, severity, verdict, and a link back to the source evidence in the repository. The same write-up is also copied into evidence/jira/DEFECT-TRIAGE.md for anyone without Jira access.

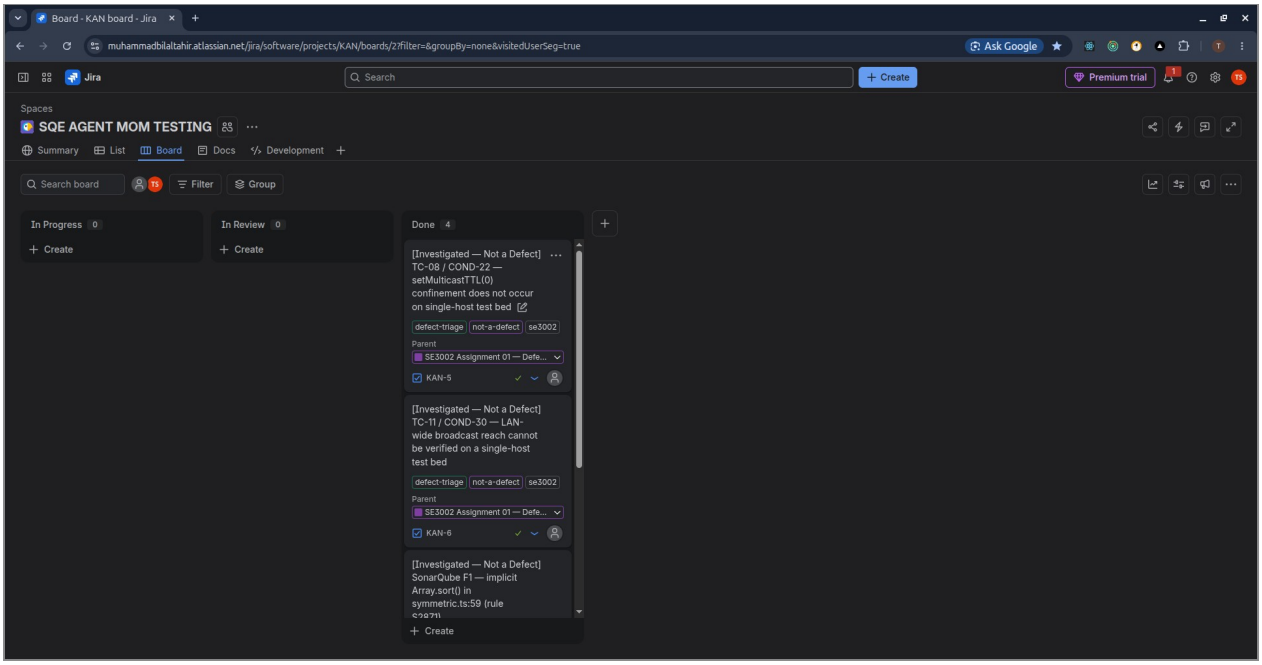

Figure 1: Jira board for project KAN (space "SQE AGENT MOM TESTING"), showing the four closed investigation records under Epic KAN-9 in the Done column: KAN-5 (TC-08), KAN-6 (TC-11), KAN-7 (SonarQube F1) and KAN-8 (SonarQube F4), each tagged defect-triage / not-a-defect / se3002. Captured 2026-09-09.

Candidate	Status	Jira issue
TC-08 (COND-22)	FAILED (expected, environment)	KAN-5 (Done, Low, not a defect)
TC-11 (COND-30)	BLOCKED (expected, precondition)	KAN-6 (Done, Low, not a defect)
SonarQube F1	Investigated	KAN-7 (Done, Low, not a defect)
SonarQube F4	Investigated	KAN-8 (Done, Low, not a defect)

0 of 4 investigated candidates met all four defect criteria: 0 open Bugs, 4 closed and fully-evidenced investigation records logged in Jira (KAN-5 through KAN-8) under Epic KAN-9.

5.4 Final Quality Judgment

Overall, the evidence supports a working implementation of the ten selected requirements. All 217 automated tests pass, with close to full line coverage. SonarQube's Quality Gate passed on the complete frozen codebase: the Reliability rating is mostly pulled down by one finding we investigated and justified, plus a spacing pattern that turned out to be cosmetic; Security is effectively clean once the one repeated address pattern is set aside. All 15 test cases were run, covering every required category, including two genuine FAILED/BLOCKED results that we investigated openly rather than hiding or manufacturing.

What's left unsupported is specific, not a general doubt about the project. NFR8 (1.2 compatibility) can't be checked against a real agentMom 1.2, because none exists and we built on Node instead of the SRS's specified Java: so the compatibility check is necessarily against our own reconstruction of that interface, which is a limitation testing alone can't close. NFR9's "all or none" delivery wording can't be turned into a clear pass/fail rate. NFR10 explicitly makes no decryption-resistance guarantee, so there was nothing to test there either. Three further assumptions are our own design choices, not SRS-derived guarantees: a fixed unicast key with no handshake (A5.2), no key rotation when a member leaves (A6.2), and the broadcast address we picked (A4.1).

The assumptions introduced during AI-assisted development shaped where we can and can't be confident. A5.2 is the most important one: a fixed key with no handshake is a real security gap. Neither SonarQube (which flagged an unrelated sort, not the missing handshake) nor our functional tests (which only confirm the key derivation is consistent, not that it's secure) can catch this, because it's a design choice, not a coding bug. This is simply something functional testing can't check.

Within the scope we actually tested: the ten requirements, on this single-host setup, against this frozen baseline: the implementation is acceptable. FR1–FR7 are backed by executed, reproducible evidence in every category the assignment asks for. NFR8–NFR10 are evaluated as far as their wording allows, with every remaining gap named rather than glossed over. This isn't a claim that the system is production-ready, genuinely compatible with agentMom 1.2, or tested at real LAN scale: those things simply weren't evaluated, and we're not pretending otherwise.